



Big Data System Design (MBAI 5110G) Winter-2026

Week 2: Big Data Storage Concept

Lectured by:

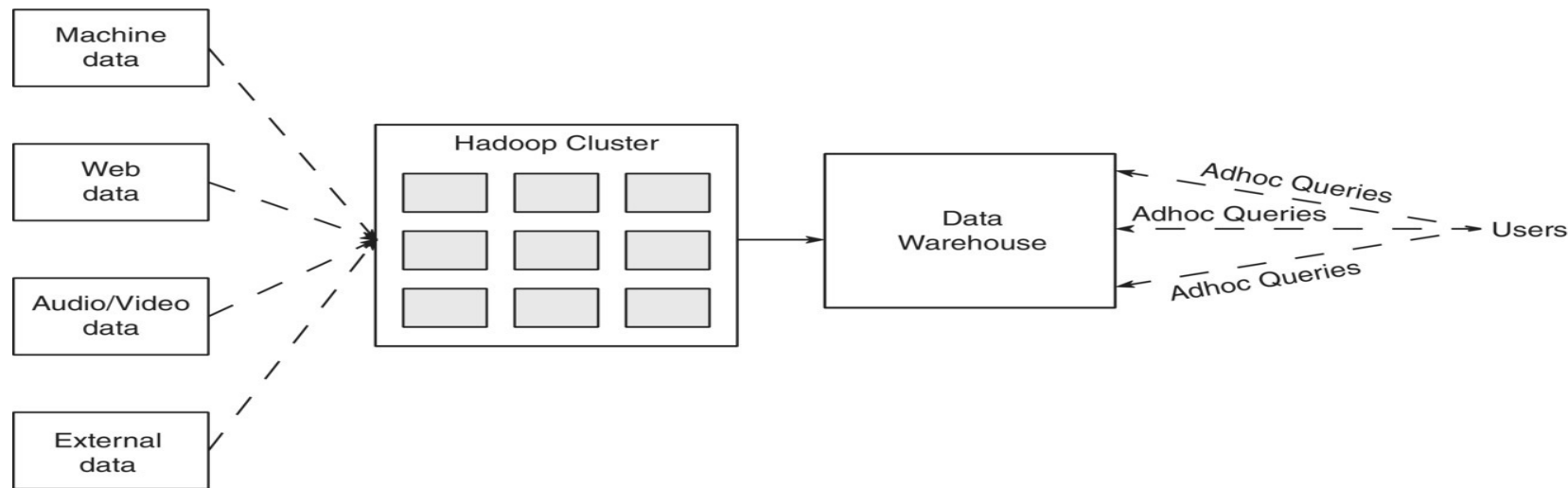
Dilli P. Sharma, PhD

Outline

- Introduction to Big Data Storage
- Cluster Computing
- Distribution Models
- Distributed File Systems (DFS)
- Relational and Non-Relational Databases
- Scaling Storages

Big Data Storage Architecture

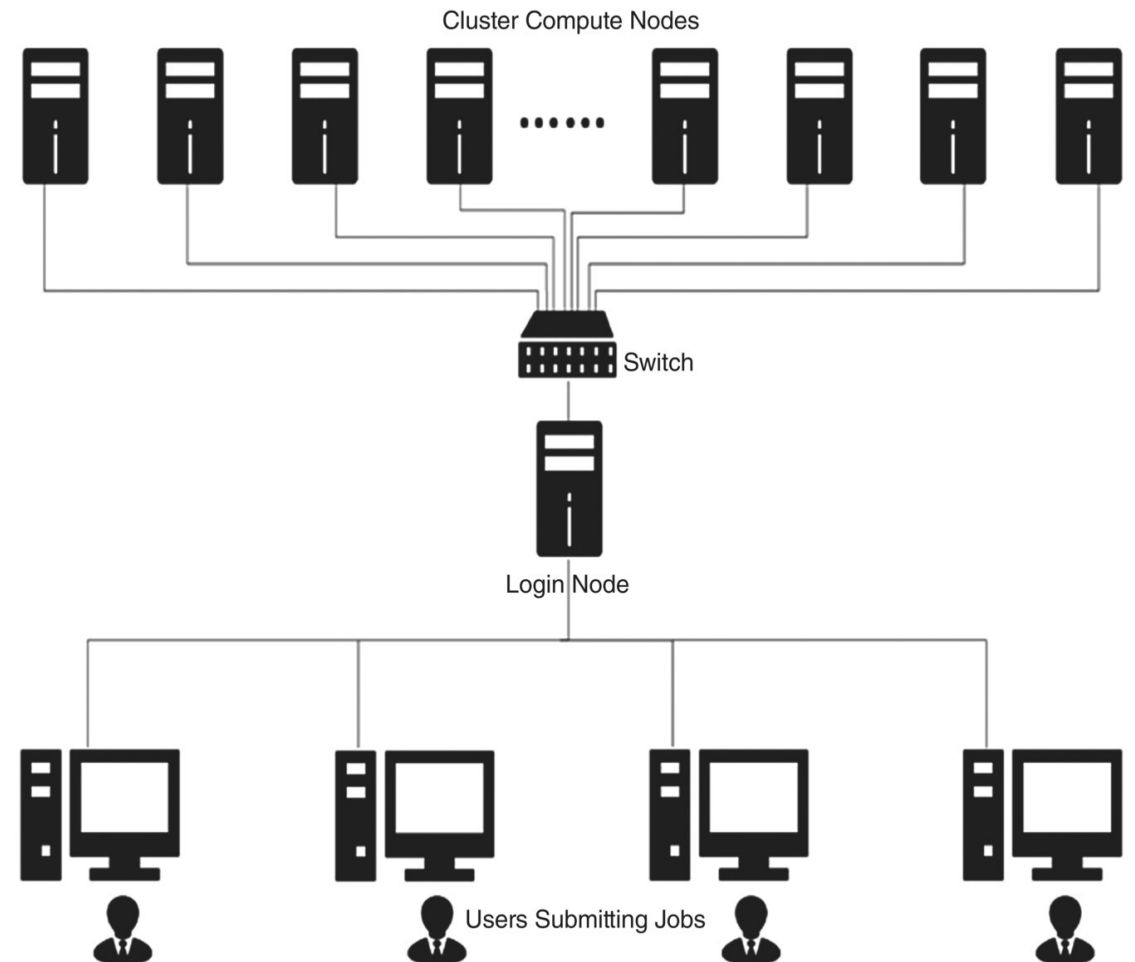
- **Architectural Transformation:** Big data storage architectures have evolved to support distributed data flow across multiple organizational structures, enabling scalable and efficient access to large volumes of data
- **Hadoop as a Central Storage Layer:** Hadoop serves as an online archive built on commodity hardware, optimized for unstructured and semi-structured data, and cost-effective for certain structured data that are expensive to store in traditional systems
- **Analytics and BI Enablement:** Data stored in Hadoop can be analyzed directly using **MapReduce**, and then integrated into data warehouses and data marts, allowing end users to query and analyze data using BI tool



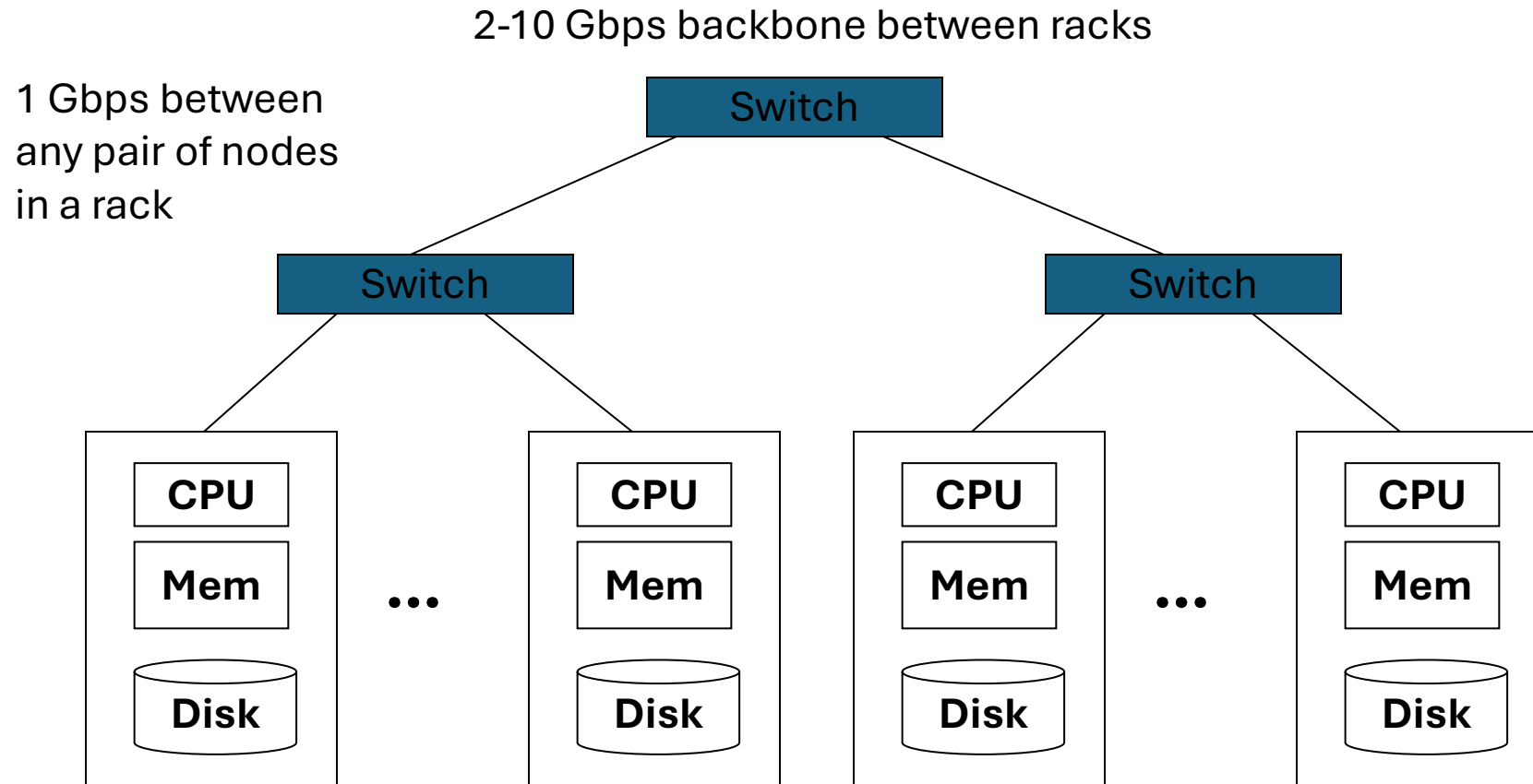
Big Data Storage Architecture

Cluster Computing

- **Cluster computing** is a distributed or parallel computing system comprising multiple stand-alone PCs connected to work as a single, integrated, highly available resource
 - It uses **commodity hardware** and supports dynamic scaling by adding or removing nodes without disrupting system operation
 - It provides **load balancing and automatic failover mechanisms**, ensuring reliable performance with minimal impact from node failures



Cluster Architecture: Examples



Each rack contains 16-64 nodes

Types of Clusters

- **High-Availability Clusters**

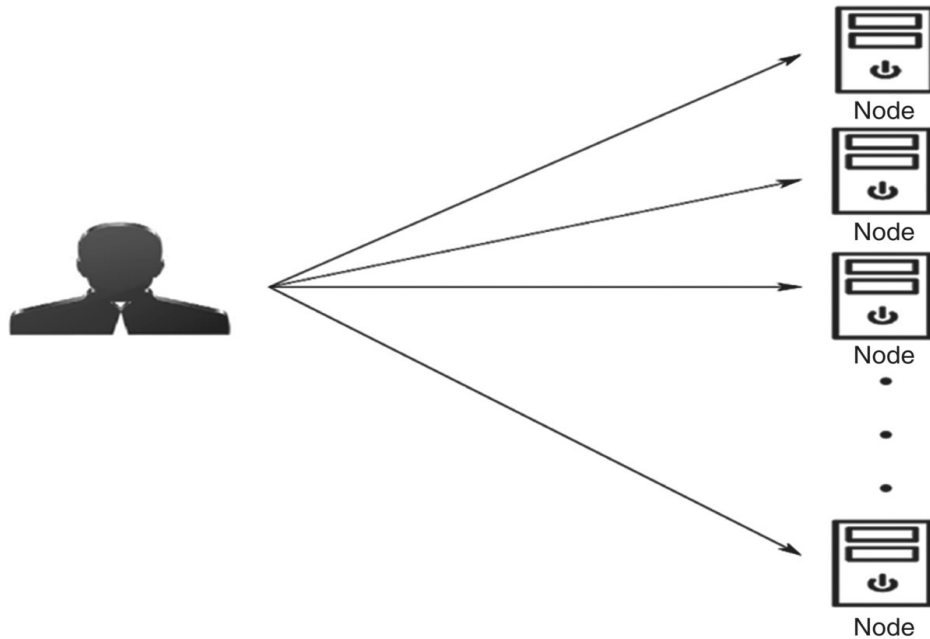
- Designed to ensure continuous system operation **by handling node failures with minimal or no downtime.**

- **Load-Balancing Clusters**

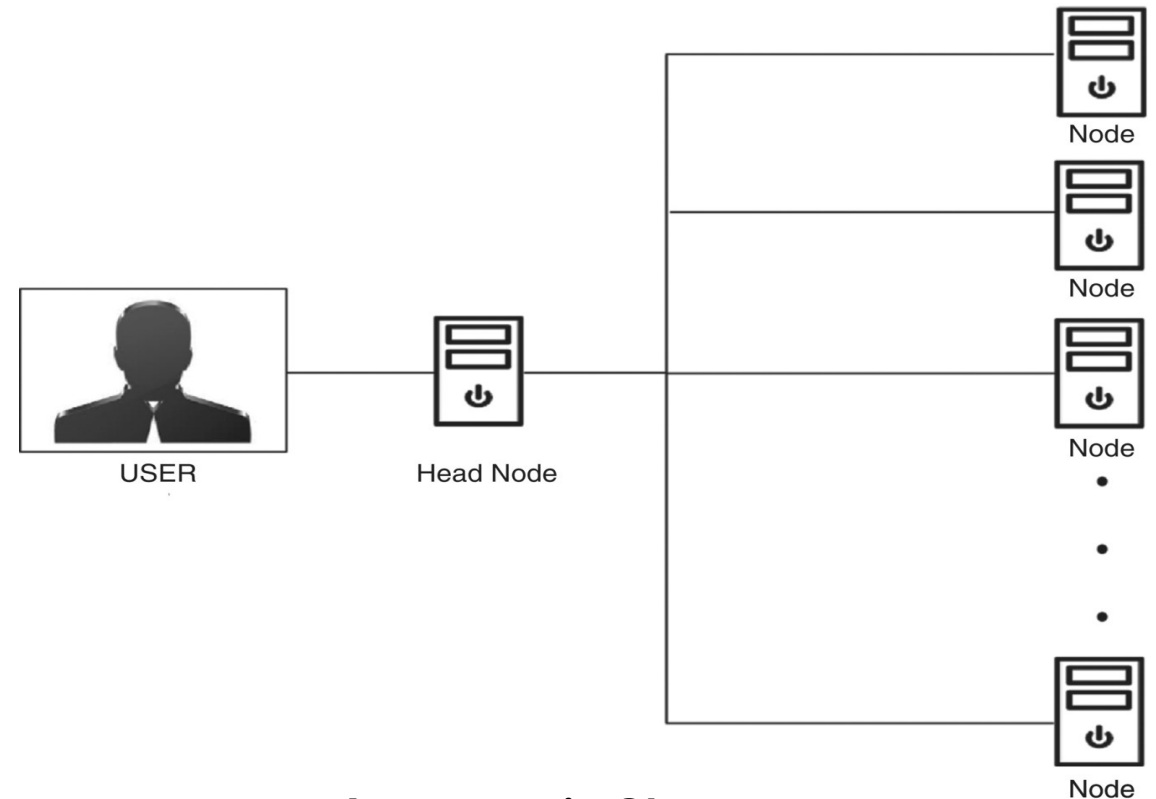
- Distribute computational workloads across nodes **to improve overall performance and resource utilization**

Cluster Structure

- **Symmetric Clusters:** All nodes are equal and can independently run applications
- **Asymmetric Clusters:** One node acts as a head/gateway node that manages access and coordination for the remaining worker nodes is simple, scalable, and easy to expand



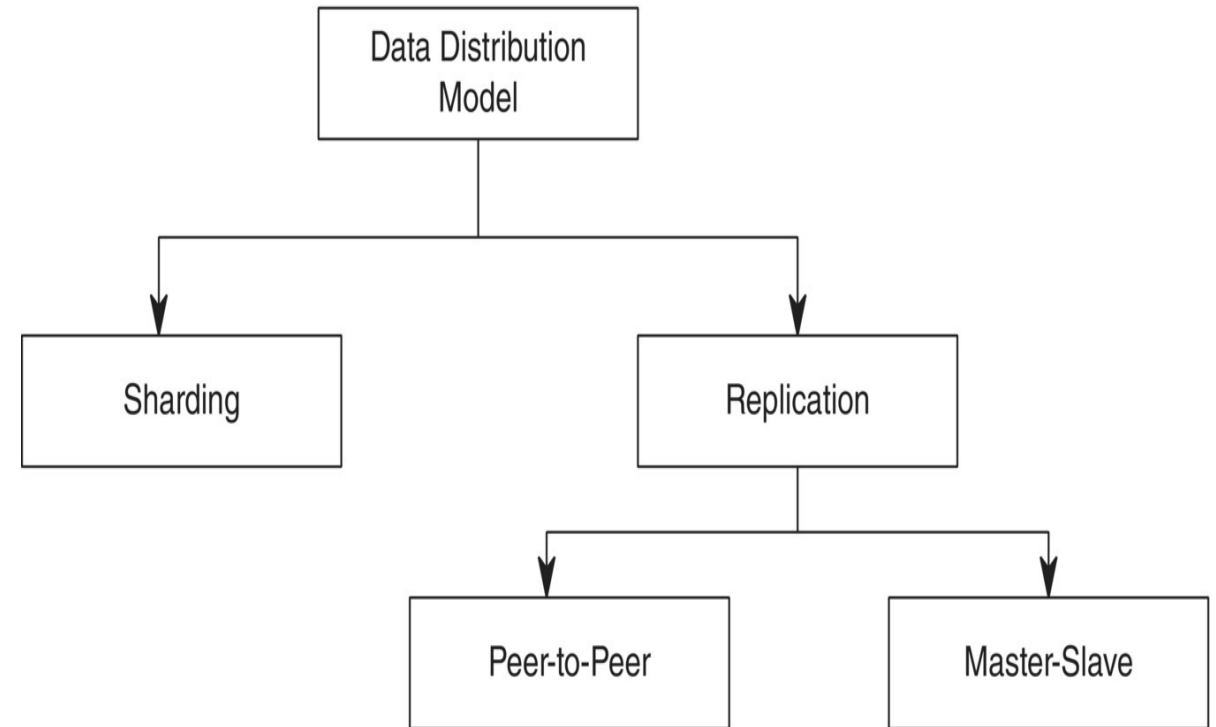
Symmetric Clusters



Asymmetric Clusters

Distribution Models

- Data is distributed across clusters to avoid expensive servers while supporting large data volumes and high read/write demands
- Distribution models enable systems to handle increased workloads and maintain availability through parallel access.
 - **Sharding**
 - **Replication**



Distribution model.

Sharding

- Sharding is the process of partitioning very large data sets into smaller and easily manageable **chunks** called **shards**
- The partitioned shards are stored by distributing them across multiple machines called **nodes**.
 - No two shards of the same file are stored in the same node
 - Each shard occupies separate nodes, and the shards spread across multiple nodes collectively constitute the datasets.

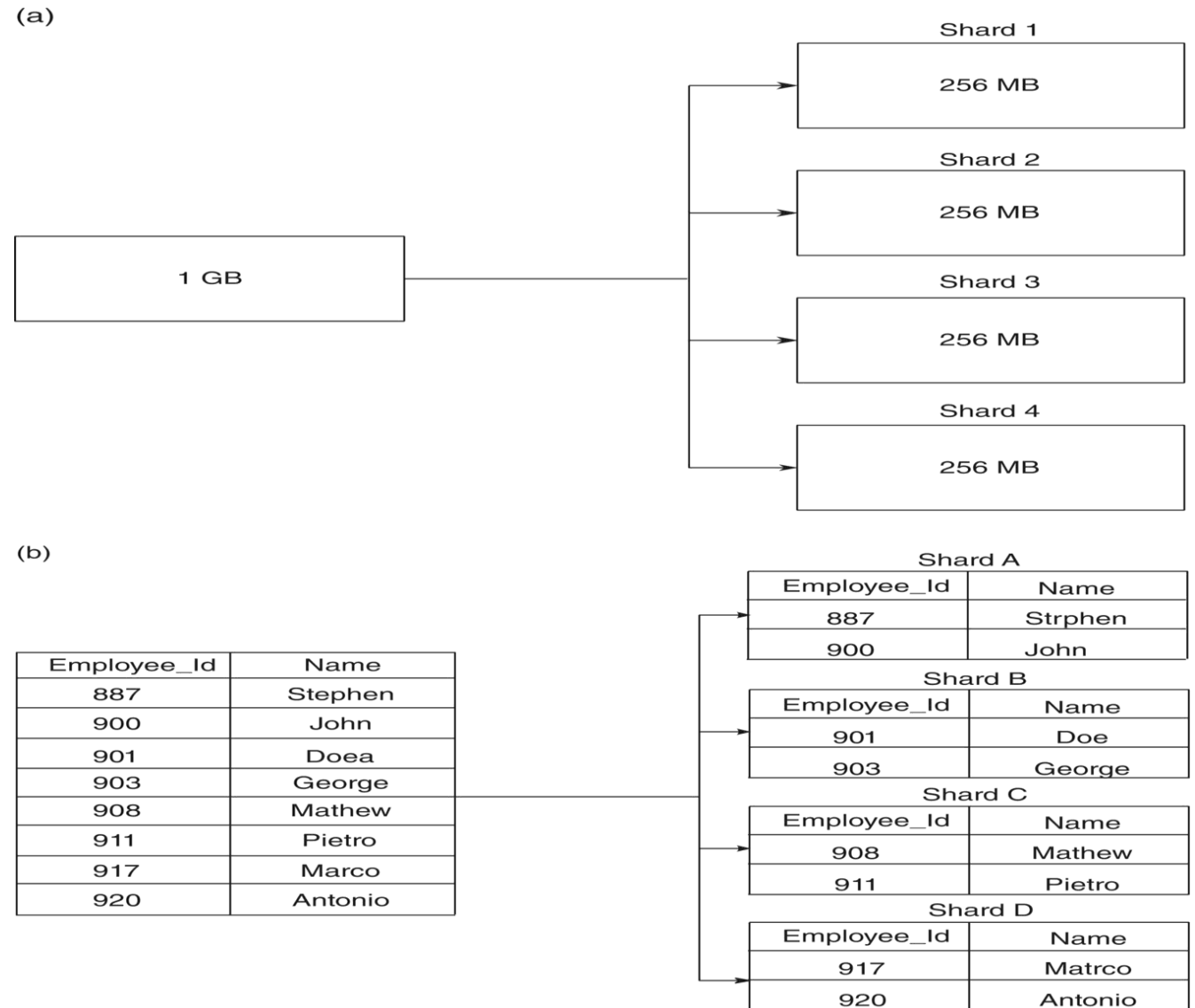
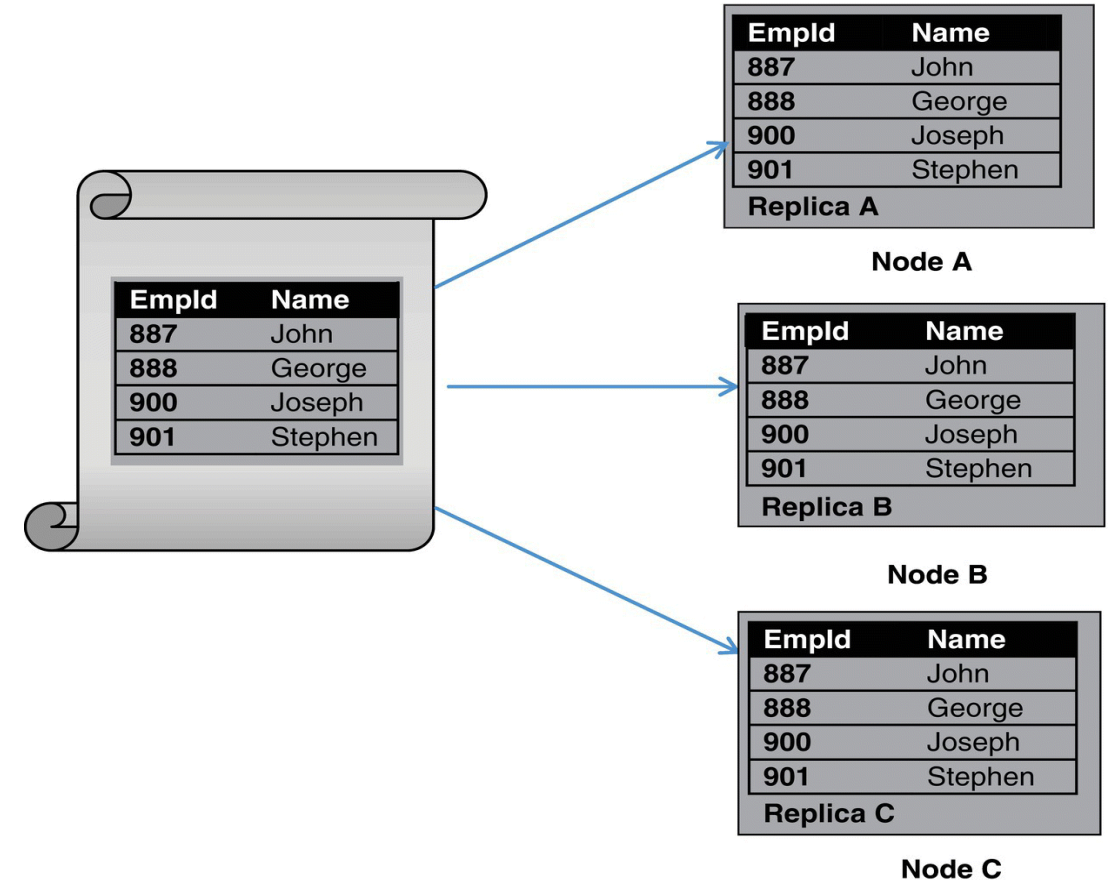


Figure: (a) Sharding. (b) Sharding example.

Data Replication

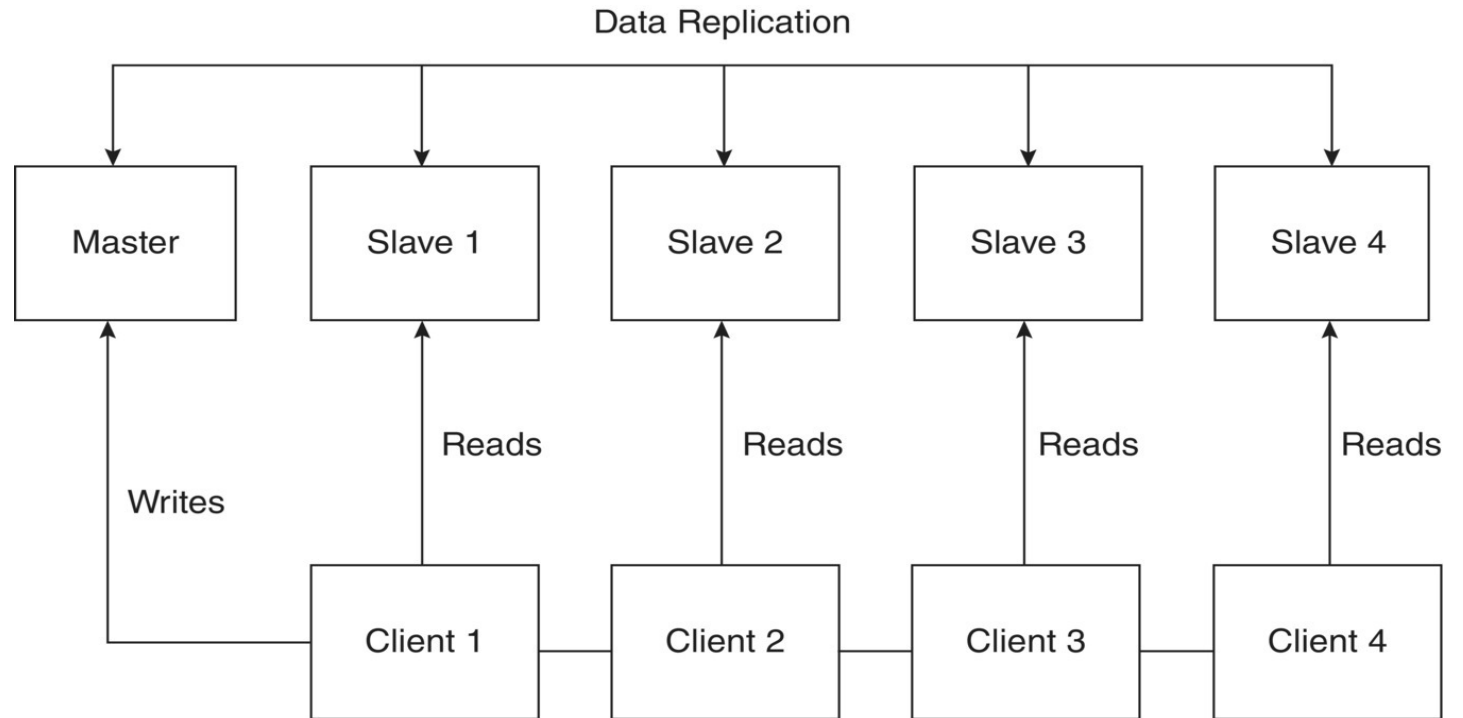
- **Data replication** creates multiple copies (replicas) of the same data on different servers in a cluster.
 - **Fault Tolerance and Availability:**
 - If a node fails or is under maintenance, replicas ensure data remains accessible without interruption
 - **Reliability of Storage:**
 - Replication prevents data loss and improves system reliability in distributed storage environments.
- **Model Types:**
 - **Master-Slave Model**
 - **Peer-to-Peer Model**



Data replication example

Master-Slave Model

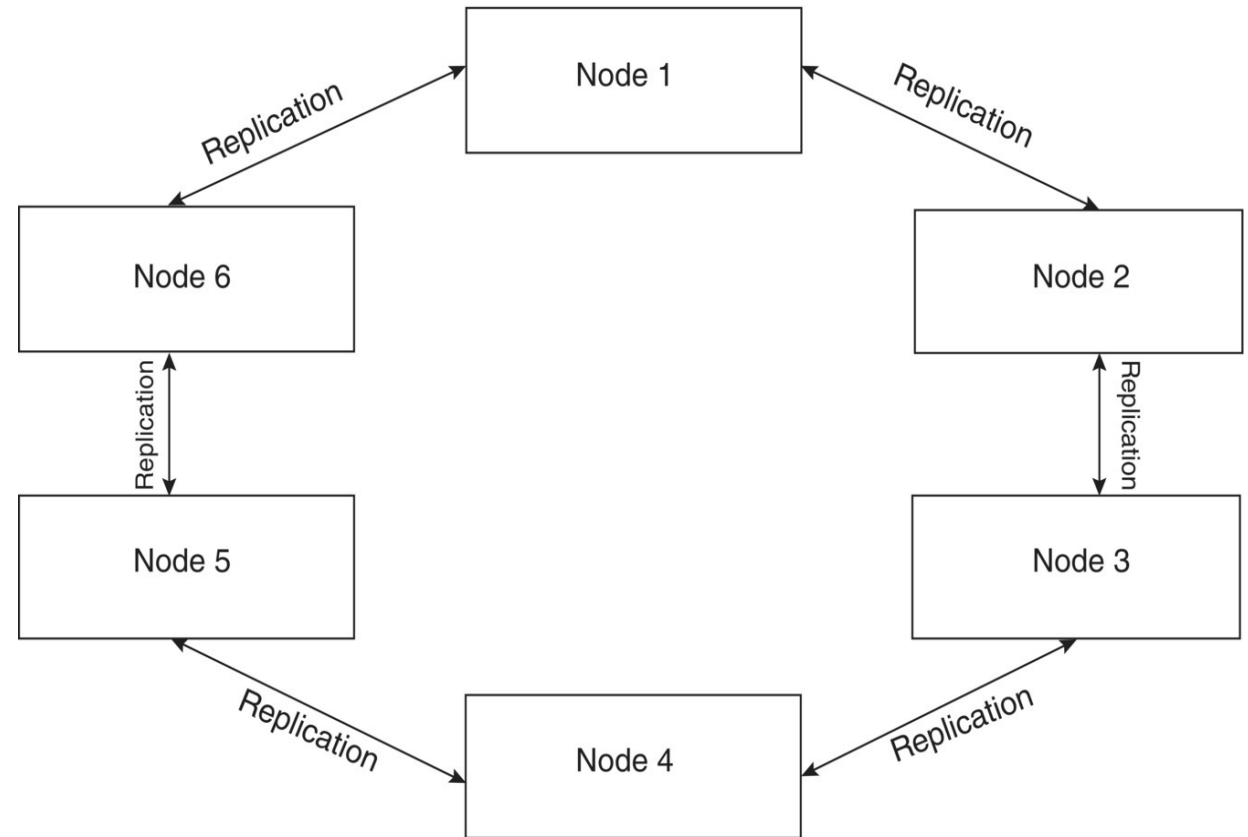
- **Centralized Control**
 - A master node handles all write operations, while slave nodes replicate data and handle read requests.
- **Efficient Read Scalability**
 - Read performance improves by adding more slave nodes to handle increasing read workloads.
- **Single Point of Failure for Writes**
 - If the master node fails, write operations stop until a new master is elected or restored



Master-Slave model.

Peer-to-Peer

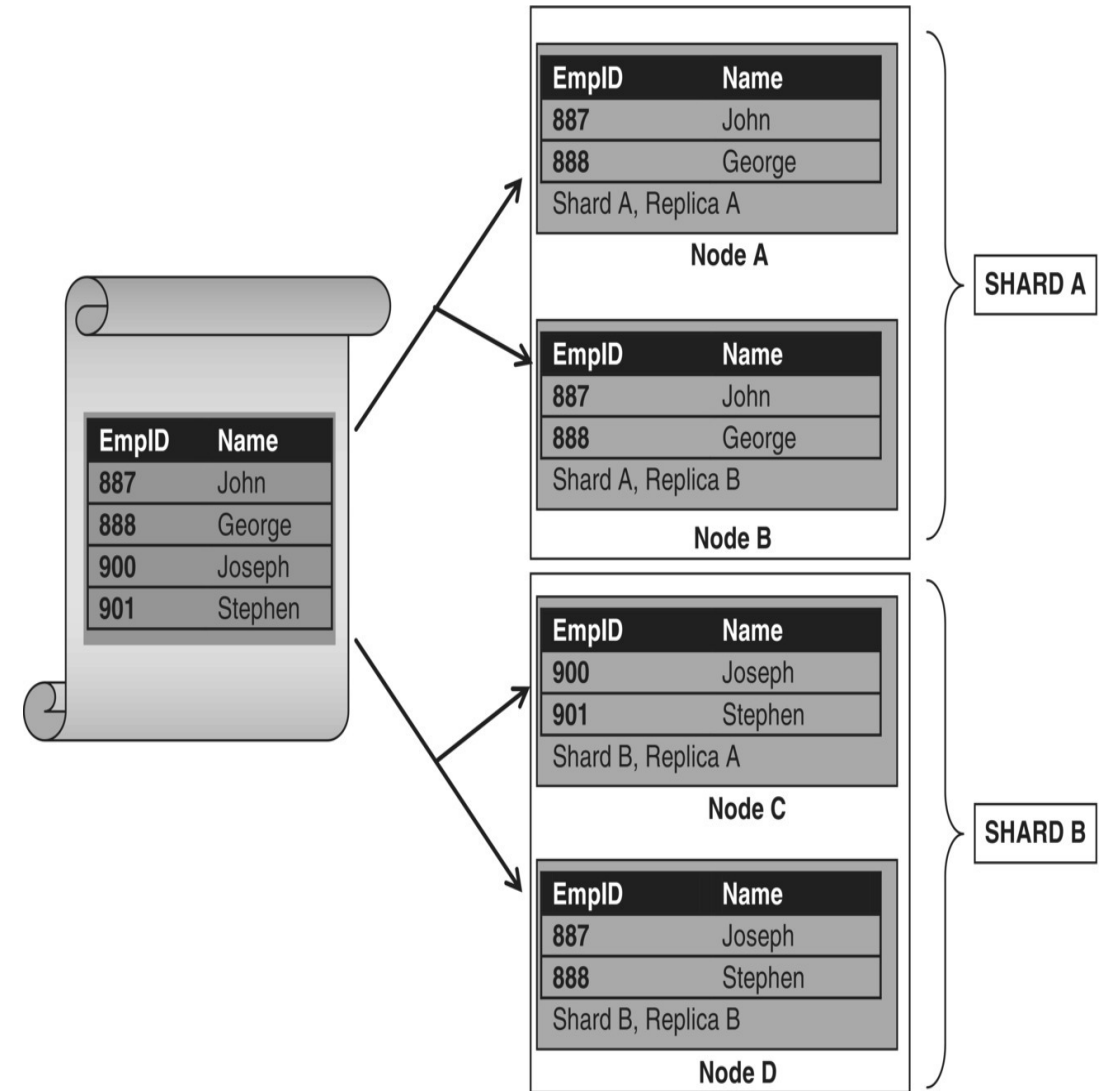
- **No Single Point of Failure**
 - Unlike master–slave, all nodes have equal responsibility, eliminating dependence on a single master node.
- **Read and Write Scalability**
 - Each node can handle both read and write requests, overcoming the write limitations of master–slave architecture.
- **Decentralized Communication**
 - Any node can initiate communication, acting as both client and server, enabling higher availability and flexibility.



Peer-to-peer model

Sharding with Replication

- **Limitation of Sharding Alone**
 - Sharding by itself provides limited fault tolerance because data is lost if a shard node fails.
- **Combined Approach for Reliability**
 - Replication is added to sharding to ensure data availability and fault tolerance when nodes go down.
- **Scalable and Highly Available Architecture**
 - Each shard is replicated across multiple nodes, enabling both horizontal scalability and high availability.



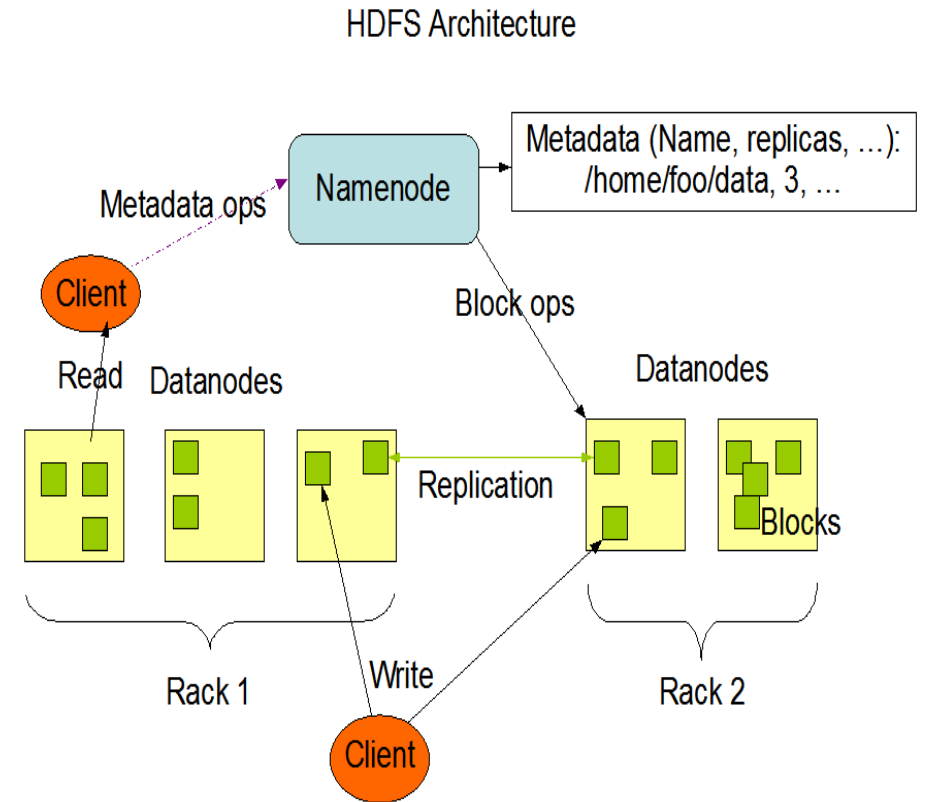
A combination of sharding and replication

Distributed File Systems (DFS)

- A **Distributed File System (DFS)** is a file storage system where data is stored across **multiple computers (nodes) in a cluster** but appears to users and applications as if it is stored on a **single local file system**.
- **Characteristics of DFS:**
 - **Data distribution** – Files are spread across many nodes
 - **Replication** – Multiple copies are stored for reliability
 - **Fault tolerance** – Data remains available even if nodes fail
 - **Scalability** – Storage grows by adding more machines
 - **High throughput** – Optimized for large-scale data access

Examples:

- Hadoop Distributed File System (HDFS)
- Google File System (GFS)
- Gluster File System
- Ceph,



Relational Databases (RDBMS)

- Data is organized into tables with rows (records) and columns (attributes)
 - Examples: Oracle, MS SQL Server, MySQL, DB2, etc.
- Poor horizontal scalability
 - RDBMS is vertically scalable and exhibits **ACID** (**a**tomicity, **c**onsistency, **i**solation, **d**urability)
- Rigid schema (no flexibility)
- High cost

Attributes/Fields

StudentName	Phone	DOB	Courseld	Faculty
James	541 754 3010	03/05/1985	1	Dr.Jeffrey
John	415 555 2671	05/01/1992	2	Dr.Lewis
Richard	415 570 2453	09/12/1999	2	Dr.Philips
Michael	555 555 1234	12/12/1995	3	Dr.Edwards
Richard	415 555 2671	02/05/1989	4	Dr.Anthony

Tuples

StudentTable

StudentId	StudentName	Phone	DOB
1615	James	541 754 3010	03/05/1985
1418	John	415 555 2671	05/01/1992
1718	Richard	415 570 2453	09/12/1999
1313	Michael	555 555 1234	12/12/1995
1718	Richard	415 555 2671	02/05/1989

RegisteredCourse

ID	Courseld	Faculty
1615	1	Dr.Jeffrey
1418	2	Dr.Lewis
1718	2	Dr.Philips
1313	3	Dr.Edwards
1819	4	Dr.Anthony

CoursesOffered

Courseld	CourseName
1	Databases
2	Hadoop
3	R Programming
4	Data Mining

Non-Relational Databases

- **Limitations of RDBMS at Scale**

- Relational databases struggle with massive volumes of unstructured and semi-structured data.

- A **NoSQL** (Not Only SQL) database includes all non-relational databases

- **Schema-Less Flexibility**

- NoSQL databases do not require fixed schemas, making them ideal for evolving data formats.

Key-value databases	Document databases	Column databases	Graph databases
Redis	MongoDB	DynamoDB	Neo4j
Riak	CouchDB	Cassandra	OrientDB
SimpleDB	RethinkDB	Accumulo	ArangoDB
BerkeleyDB Oracle	MarkLogic	Big Table	FlockDB

ACID vs BASE Model (RDBMS vs NoSQL)

- **RDBMS follows ACID:**
 - Ensures strong consistency, atomicity, isolation, and durability for transactions.
- **NoSQL follows CAP & BASE:**
 - Designed around **Consistency, Availability, and Partition Tolerance (CAP)** and the **BASE** (basically, available, soft state, eventually consistent) model.
- **Eventual Consistency:**
 - NoSQL systems provide high availability and scalability by allowing data to become consistent over time.

NewSQL Databases

- **NewSQL** databases provide scalable performance similar to that of **NoSQL systems**, combining the **ACID properties** of a traditional database management system
- Some examples of NewSQL Databases are:
 - **Clustrix**
 - High-performance, fault-tolerant distributed database
 - Designed for **massive, high transactional volume** applications
 - **NuoDB**
 - Cloud-based, scale-out, fault-tolerant distributed database
 - Supports **batch and real-time SQL queries**
 - **VoltDB**
 - In-memory, scale-out, fault-tolerant distributed database
 - Optimized for **real-time decision-making**
 - **MemSQL**
 - In-memory, high-performance, fault-tolerant distributed database
 - Known for **blazing fast performance** for **real-time analytics**

Scaling Storage

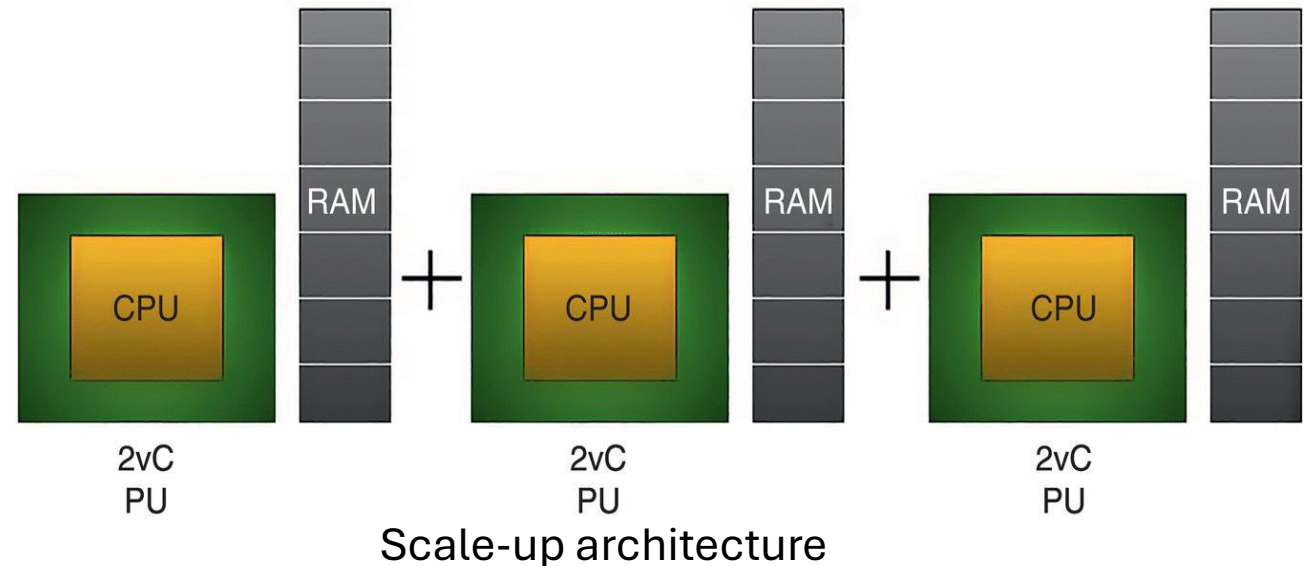
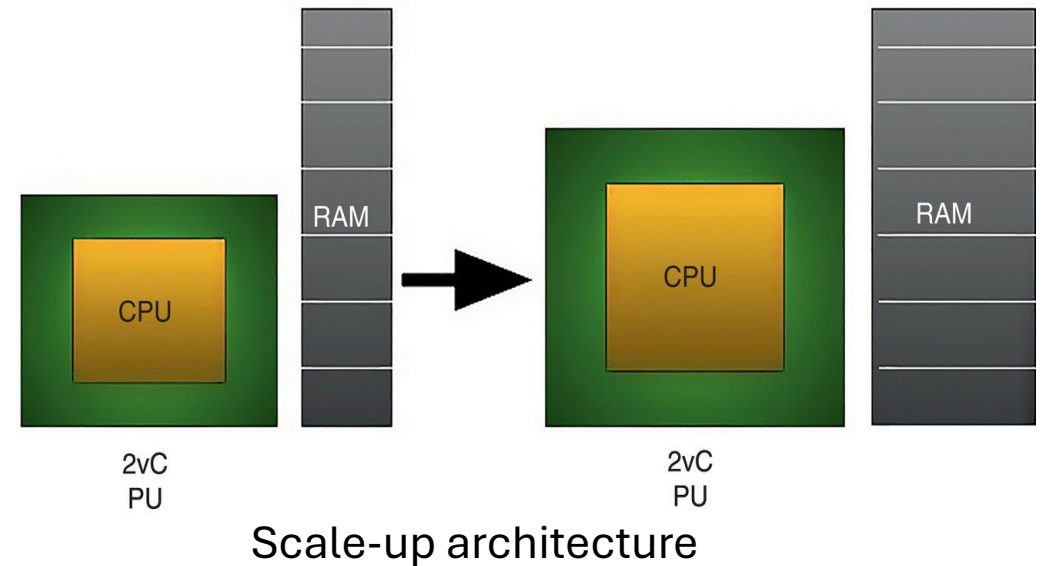
- Ability of a system to meet increasing storage demand while maintaining performance and efficiency

- **Scaling-Up (Vertical Scaling)**

- Adds more resources (CPU, RAM, storage) to a single server
- Limited by the maximum capacity of the server
- Example: Upgrading RAM from 32 GB → 128 GB

- **Scaling-Out (Horizontal Scaling)**

- Adds new servers or nodes to the system
- Uses low-cost commodity hardware
- Enables big data systems to scale seamlessly and handle petabytes of data



Reference/textbook

- Chapter 2-Big Data Storage Concept (Textbook): Big Data: Concepts, Technology, and Architecture, Balamurugan Balusamy, Nandhini Abirami R, Seifedine Kadry, Amir H. Gandomi, Wiley, 2021
- HDFS Architectural Guide:
https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html

Next Session: NoSQL Database

Q/A

Thank you!